Jacobian Descent for Multi-Objective Optimization

Pierre Quinton* (LTHI, EPFL) Valérian Rey* (SimplexLab)
*Equal contribution

Paper · Code

Gradient descent optimizes one objective. Jacobian descent optimizes many at once.

Our aggregator, **UPGrad**, never conflicts with any objective and provably converges to the **Pareto front**.

Jacobian descent

Training with several losses usually means **summing them** and running SGD. This can silently **sacrifice some objectives** for others. Instead, we minimize $\boldsymbol{f} : \mathbb{R}^n \to \mathbb{R}^m$ directly, using its **Jacobian**, whose rows are the gradients:

$$\mathcal{J}\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \nabla f_1(\boldsymbol{x})^\top \\ \vdots \\ \nabla f_m(\boldsymbol{x})^\top \end{bmatrix} \in \mathbb{R}^{m \times n}$$

An **aggregator** $\mathcal{A} : \mathbb{R}^{m \times n} \to \mathbb{R}^n$ reduces it to a single update:

Gradient descent	Jacobian descent
$\boldsymbol{x} \leftarrow \boldsymbol{x} - \eta \nabla f(\boldsymbol{x})$	$\boldsymbol{x} \leftarrow \boldsymbol{x} - \eta \mathcal{A}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x}))$

GD is the special case $m = 1, \mathcal{A} = \mathrm{id}$. Many multi-task learning methods (MGDA, PCGrad, CAGrad, Nash-MTL, …) are **just JD with a different aggregator**.

Stochastic variants. Aggregate an *estimate* of the Jacobian (SEJD), or a random *subset of its rows* (stochastic sub-Jacobian descent, SSJD), as SGD does with gradients.

What makes a good aggregator?

Non-conflicting. $0 \preceq J\,\mathcal{A}(J)$
No objective increases, to first order. $\mathcal{A}(J)$ lies in the *dual cone* of the gradients.

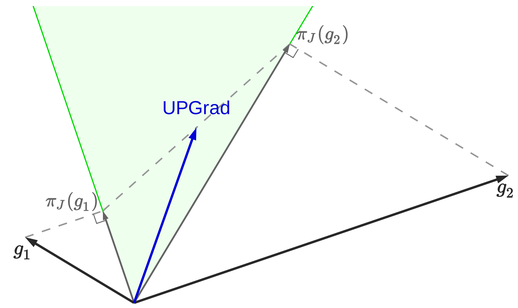
Linear under scaling. $\boldsymbol{c} \mapsto \mathcal{A}\big(\mathrm{diag}(\boldsymbol{c})\, J\big)$ is linear for $\boldsymbol{c} \succ 0$
Each gradient contributes in proportion to its norm: small gradients can neither dominate nor cancel the update.

Weighted. $\mathcal{A}(J) = J^\top \boldsymbol{w}$ for some $\boldsymbol{w} \in \mathbb{R}^m$
The update stays in the span of the gradients, where the first-order approximation is most accurate.

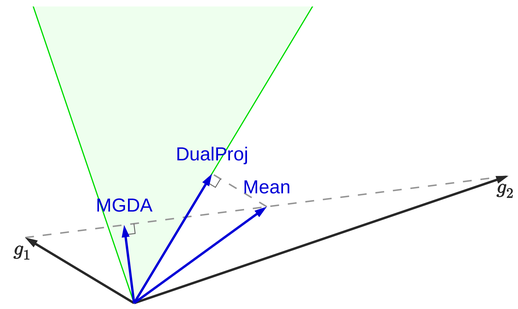
Aggregator	Non-conflicting	Linear under scaling	Weighted
$\mathcal{A}_{\text{Mean}}$	✗	✓	✓
$\mathcal{A}_{\text{MGDA}}$	✓	✗	✓
$\mathcal{A}_{\text{DualProj}}$	✓	✗	✓
$\mathcal{A}_{\text{PCGrad}}$	✗	✓	✓
$\mathcal{A}_{\text{GradDrop}}$	✗	✗	✗
$\mathcal{A}_{\text{IMTL-G}}$	✗	✗	✓
$\mathcal{A}_{\text{CAGrad}}$	✗	✗	✓
$\mathcal{A}_{\text{RGW}}$	✗	✓	✓
$\mathcal{A}_{\text{Nash-MTL}}$	✓	✗	✓
$\mathcal{A}_{\text{Aligned-MTL}}$	✗	✗	✓
$\mathcal{A}_{\text{UPGrad}}$ **(ours)**	✓	✓	✓

UPGrad: Unconflicting Projection of Gradients

Project each gradient onto the dual cone (green), then average.



(a) $\mathcal{A}_{\text{UPGrad}}$ never conflicts with $\boldsymbol{g}_1$ or $\boldsymbol{g}_2$.



(b) $\mathcal{A}_{\text{Mean}}$ conflicts with $\boldsymbol{g}_1$. $\mathcal{A}_{\text{DualProj}}$ lands on the cone's boundary. $\mathcal{A}_{\text{MGDA}}$ is almost orthogonal to $\boldsymbol{g}_2$ because of its larger norm.

$$\mathcal{A}_{\text{UPGrad}}(J) = \frac{1}{m}\sum_{i=1}^{m} \pi_J(\boldsymbol{g}_i), \qquad \pi_J(\boldsymbol{x}) = \operatorname*{arg\,min}_{\boldsymbol{y}:\, \mathbf{0} \preceq J\boldsymbol{y}} \|\boldsymbol{y} - \boldsymbol{x}\|^2$$

Dual formulation. For any $\boldsymbol{u} \in \mathbb{R}^m$, $\pi_J(J^\top \boldsymbol{u}) = J^\top \boldsymbol{w}$ with

$$\boldsymbol{w} \in \operatorname*{arg\,min}_{\boldsymbol{v}:\, \boldsymbol{u} \preceq \boldsymbol{v}} \boldsymbol{v}^\top J J^\top \boldsymbol{v}$$

an m-dimensional QP, independent of n. Taking $\boldsymbol{u} = \boldsymbol{e}_i$ gives weights $\boldsymbol{w}_i$, and

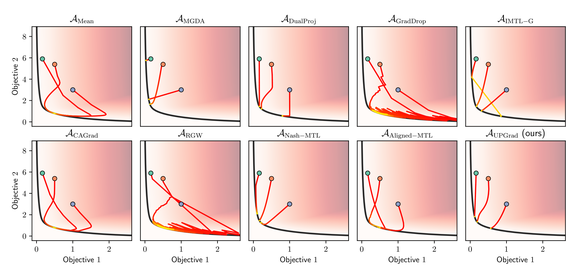
$$\mathcal{A}_{\text{UPGrad}}(J) = J^\top \Big(\frac{1}{m}\sum_{i=1}^{m} \boldsymbol{w}_i\Big)$$

- Satisfies **all three properties**.
- If no two gradients conflict, it is simply the **mean**.

Convergence to the Pareto front

Theorem. Let $\boldsymbol{f}$ be β-smooth and $\preceq$-convex, with a compact Pareto front. JD with $\mathcal{A}_{\text{UPGrad}}$ and step size $\eta = \frac{1}{\beta\sqrt{m}}$ (with bounded weights) makes $\boldsymbol{f}(\boldsymbol{x}_t)$ **converge to the Pareto front**.

Usual guarantees only reach a *weakly Pareto stationary* point, which can be far from optimal even for convex $\boldsymbol{f}$. $\mathcal{A}_{\text{UPGrad}}$ is the **first non-conflicting aggregator** proven to reach the Pareto front in this setting.



Trajectories in objective space on a convex quadratic $\boldsymbol{f} : \mathbb{R}^2 \to \mathbb{R}^2$ (black: Pareto front).

Gramian-based Jacobian descent

For **all weighted aggregators of the table**, the weights $\mathcal{W}(G)$ only depend on the Gramian $G = JJ^\top$, so

$$\mathcal{A}(\mathcal{J}\boldsymbol{f}(\boldsymbol{x})) = \nabla\Big(\mathcal{W}(\mathcal{G}\boldsymbol{f}(\boldsymbol{x}))^\top \boldsymbol{f}\Big)(\boldsymbol{x})$$

A step needs only the $m \times m$ **Gramian** and **one backward pass**. Like gradients, Gramians can be backpropagated, **without ever storing the Jacobian**:

(chain rule) $\mathcal{G}(\boldsymbol{f} \circ \boldsymbol{g})(\boldsymbol{x}) = \mathcal{J}\boldsymbol{f}(\boldsymbol{g}(\boldsymbol{x}))\; \mathcal{G}\boldsymbol{g}(\boldsymbol{x})\; \mathcal{J}\boldsymbol{f}(\boldsymbol{g}(\boldsymbol{x}))^\top$

(sum rule) $\mathcal{G}\boldsymbol{f}(\boldsymbol{x}_1, \dots, \boldsymbol{x}_k) = \sum_{i=1}^{k} \mathcal{G}_{\boldsymbol{x}_i}\boldsymbol{f}(\boldsymbol{x}_1, \dots, \boldsymbol{x}_k)$

Memory: $\mathcal{O}(mn) \;\to\; \mathcal{O}(n + m^2)$

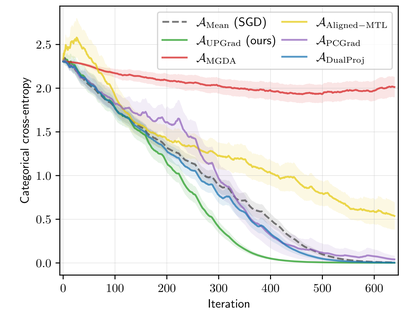
87–93% less memory **42–89% faster**

Instance-wise risk minimization

Treat the **loss of each training example as its own objective**:

$$\underbrace{\bar{f}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m} f_i(\boldsymbol{\theta})}_{\text{ERM}} \quad \longrightarrow \quad \underbrace{\boldsymbol{f}(\boldsymbol{\theta}) = \big[f_1(\boldsymbol{\theta}) \;\cdots\; f_m(\boldsymbol{\theta})\big]^\top}_{\text{IWRM (ours)}}$$

Optimized with SSJD on mini-batches. With $\mathcal{A}_{\text{Mean}}$, this is exactly SGD.



CIFAR-10 (1024 images, batch size 32): $\mathcal{A}_{\text{UPGrad}}$ **minimizes the average training loss faster than SGD**, the method that directly targets it.

Test acc. (%)	SVHN	CIFAR-10	EuroSAT	MNIST	F-MNIST	K-MNIST
$\mathcal{A}_{\text{Mean}}$ (SGD)	69.6	47.7	68.5	**96.3**	80.7	76.9
$\mathcal{A}_{\text{UPGrad}}$	**69.7**	**48.1**	**73.0**	96.0	80.8	**77.1**
$\mathcal{A}_{\text{DualProj}}$	69.3	47.2	72.7	96.1	80.8	76.4
$\mathcal{A}_{\text{GradDrop}}$	68.9	46.8	71.9	**96.3**	**80.9**	76.9
$\mathcal{A}_{\text{MGDA}}$	41.7	32.9	62.5	85.2	73.4	53.7

TorchJD: JD in PyTorch

```
pip install "torchjd[quadprog_projector]"
```

```python
from torchjd import autojac
from torchjd.aggregation import UPGrad

loss_fn = CrossEntropyLoss(reduction="none")
for input, target in loader:
    losses = loss_fn(model(input), target)
    autojac.backward(losses)
    autojac.jac_to_grad(model.parameters(), UPGrad())
    optimizer.step()
    optimizer.zero_grad()
```

github.com/SimplexLab/TorchJD